Collaboration, Quality, Test
WS2025/2026
Prof. Dr. Jóakim von Kistowski

Exercise 7 – Display Sudoku in a Web Browser

Learning objectives: Using Maven to import a lot of libraries and create a runnable Jar

Today, we create a tiny web server to display our Sudoku field in a browser.

The main parts of this exercise are:

1. Create a String representation of a Sudoku field
2. Import the Javalin Framework in your Sudoku Project and print out the Sudoku representation over HTTP
3. Create a "fat" Jar of your application

Part 1 – Create a String Representation for your Sudoku

1. In your *SudokuCreator* class implement the following method:
   `public String boardToString(int[][] board)`
   The method takes a Sudoku board as a two-dimensional integer array and creates a multiline string representation.
2. In *SudokuCreatorTest*, add unit tests that test your new method.

Part 2 – Serve Sudoku over HTTP

The goal of this part is to extend your code so that you can view a newly created Sudoku field in a browser.
To do this, we use the Javalin web framework. Javalin is a relatively simple framework that can serve content over HTTP.

1. Read up on how to include Javalin in your project in the relevant documentation: https://javalin.io/tutorials/maven-setup
2. Import Javalin into your project and refresh your Maven dependencies
3. Create a class with the name *SudokuApplication* in the *de.thab.cqt.sudoku* package in *src/main/java*
4. In *SudokuApplication*, create a method do instantiate your SudokuSolver and SudokuCreator. Configure Javalin so that it serves a new Sudoku board when accessing your web server using an HTTP GET request at Port 8080 with the URL "/".
   Hint: Serve the board as a plain-text response first, don't use fancy HTML (this is the Javalin default behavior). The example example in the Javalin Maven setup documentation is pretty close to what you need.

5. Create a main method that runs your Javalin code.
6. Run your code. Open your browser and point it to: http://localhost:8080/
   Your Sudoku should be displayed.
7. ***Optional***: Rework your code to display everything in a fancy HTML page.

Part 3 – Packaging Everything in an executable Jar

We want to package our code in a single Jar that can be run from the command line using `java -jar <jar_name>.jar`. To do so, we need to do two things:

- Configure the Jar so that our main-method runs when the Jar is being executed
- Package all used libraries into the Jar so that it will still run after being distributed

1. In the pom.xml add a <build> section with a nested <plugins> section. Your pom.xml should look something like this:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">

    .....

    <dependencies>

        .......

    </dependencies>
    <build>
        <plugins>

        </plugins>
    </build>
</project>
```

2. Search for the Maven Jar Plugin online. Add it to the plugin configurations. Configure it so that it knows the location of your class containing the main-method, adding it to the Jar's manifest.
3. Search for the Maven Shade Plugin online. Add it to your plugin configurations. Configure it so that it creates a "fat" Jar, containing all dependencies, when packaging your Maven project.
   Hint: The maven-shade-plugin creates a temporary pom.xml with the name dependency-reduced-pom.xml. Consider adding this file to your .gitigore if you use Git.